
2373 - MODULE 2

TEXT PREPROCESSING

Converting Language into Mathematics:
From Counts to Context.

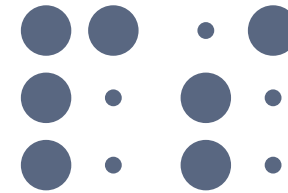
Section 1: The Raw Material – Text Preprocessing



TEXT REPRESENTATION & PREPROCESSING



How Machines "Read" – From Cleaning
Text to Vector Spaces



Imagine: A sentence passing through a
prism and turning into a stream of
binary/vectors.

THE FUNDAMENTAL PROBLEM



Computers don't speak English. They speak Math (Matrix operations).



The Challenge: How do we convert variable-length, messy text strings into fixed-size numerical vectors?



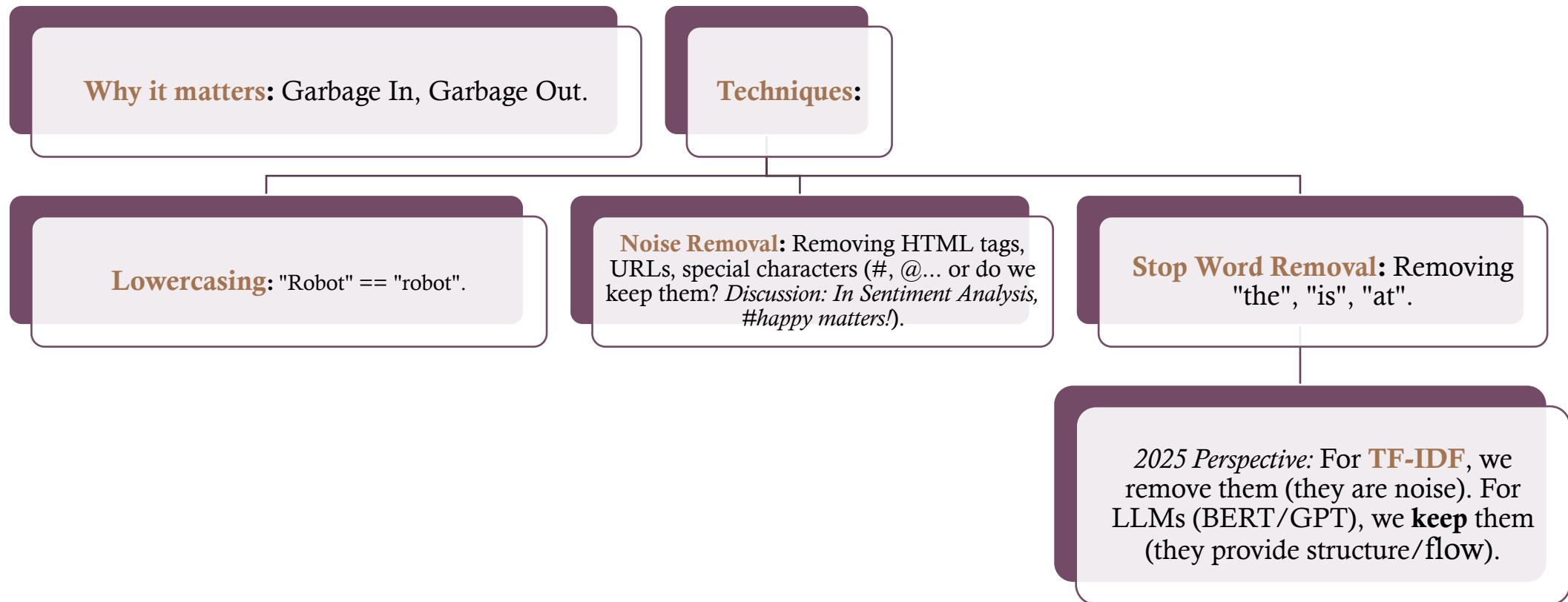
The Pipeline:

Clean: Remove noise (Preprocessing).

Tokenize: Chop into units.

Vectorize: Turn units into numbers.

STEP 1: **CLEANING** & NORMALIZATION (THE HYGIENE)

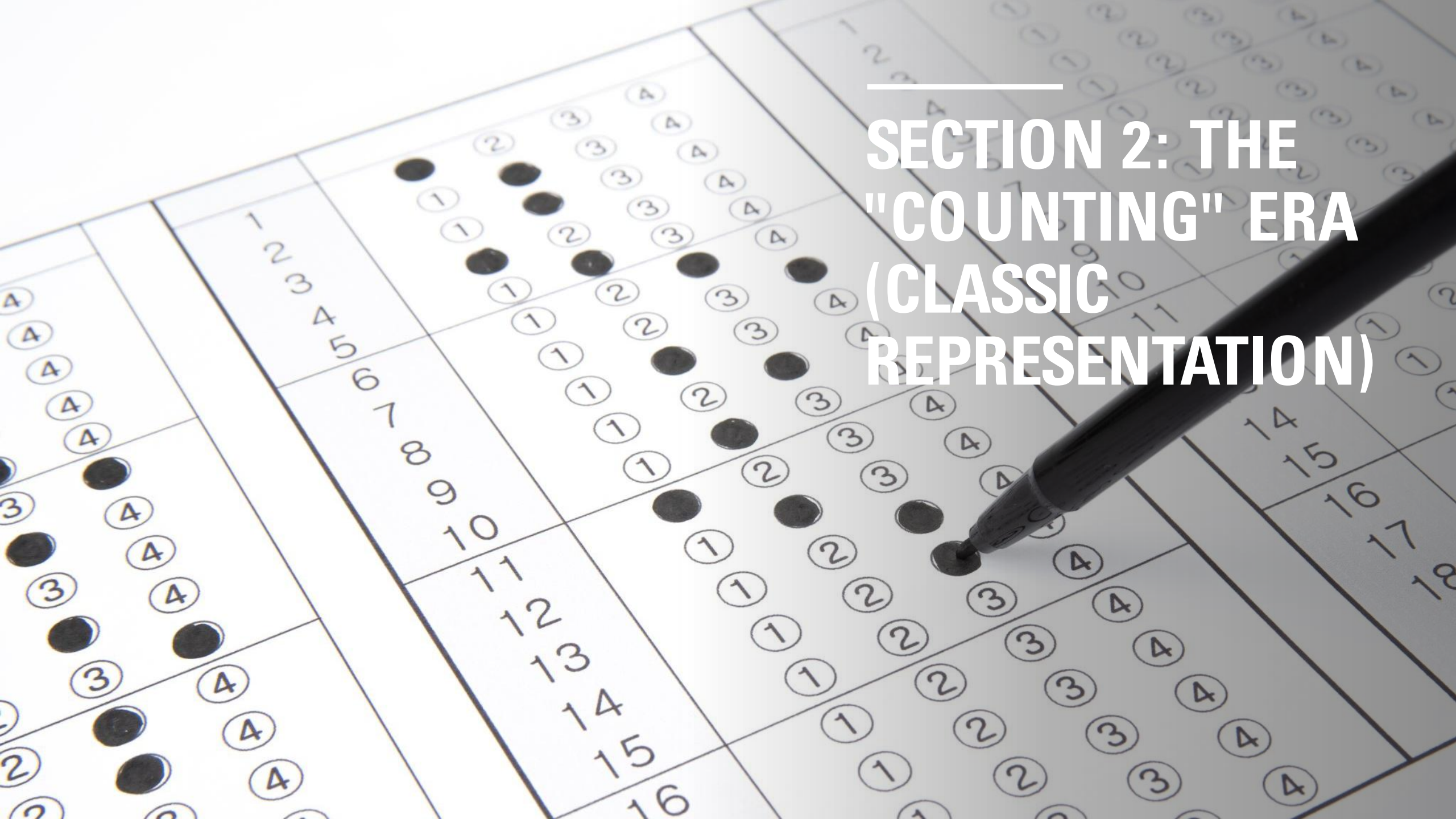


STEP 2: TOKENIZATION

- **Definition:** Breaking text into smaller units (Tokens).
- **Types:**
 - **Word-Level:** ['I', 'love', 'AI'] (NLTK/SpaCy approach).
 - **Sub-word Level (Modern):** ['Un', 'friend', 'li', 'ness'] (BERT/GPT approach).
- **Robotics Context:** If a robot knows "walk", does it know "walking"? Sub-word tokenization helps it understand they are related.

STEP 3: **STEMMING** VS. **LEMMATIZATION**

- **The Goal:** Reducing words to their base *root* to group similar meanings.
 - **Stemming (The Hatchet):** Chops off ends.
 - *Example:* "Running", "Runner" -> "Run".
 - *Fast, but crude.* (Can produce non-words).
 - **Lemmatization (The Scalpel):** Uses a dictionary/morphology to find the root.
 - *Example:* "Better" -> "Good".
 - *Slower, but precise.*
 - **Lab Prep:** We will use SpaCy (Lemmatization) vs NLTK (Stemming) to see the difference.
-

A close-up photograph of a 10-frame grid, a common tool for teaching basic arithmetic. The grid is divided into four columns and four rows. The first column contains numbers 1 through 4, the second 5 through 8, the third 9 through 12, and the fourth 13 through 16. Each number is inside a circle. A black pen is pointing to a dot in the circle containing the number 15. Other dots are visible in the circles for 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, and 16. The background is a light gray.

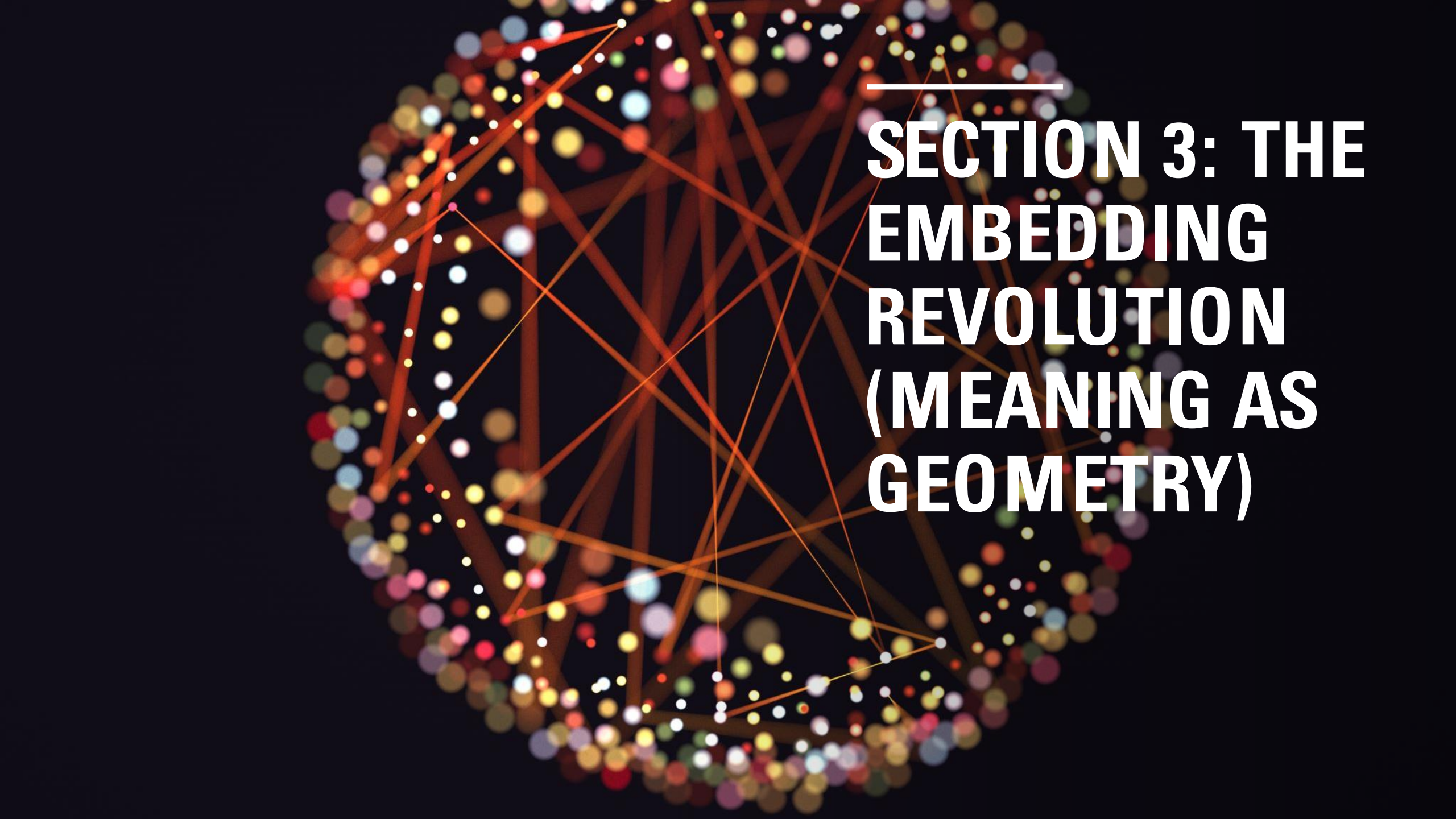
SECTION 2: THE "COUNTING" ERA (CLASSIC REPRESENTATION)

REPRESENTATION 1: BAG OF WORDS (BOW)

- **Concept:** Disregard grammar and order; just count the frequency.
 - **Visual:** A sentence "The robot fixed the car" thrown into a bag.
 - Vector: {'the': 2, 'robot': 1, 'fixed': 1, 'car': 1}
 - **The Flaw:**
 - "The robot killed the human" vs. "The human killed the robot."
 - BoW sees these as *identical* vectors. **Meaning is lost without order.**
-

REPRESENTATION 2: TF-IDF

- **Term Frequency - Inverse Document Frequency.**
 - **The Intuition:**
 - If a word appears often in *this* document -> It is important (**TF**).
 - If a word appears often in *all* documents (like "the") -> It is unimportant (**IDF**).
 - **The Result:** A weighted vector where rare, unique words (like "microcontroller") have high scores.
 - **Use Case 2025:** Still used in simple search algorithms and keyword extraction before feeding data to an LLM.
-



SECTION 3: THE EMBEDDING REVOLUTION (MEANING AS GEOMETRY)

THE SHIFT TO "EMBEDDINGS" (2013)



Concept: Instead of counting words, let's represent them as coordinates in a multi-dimensional space.



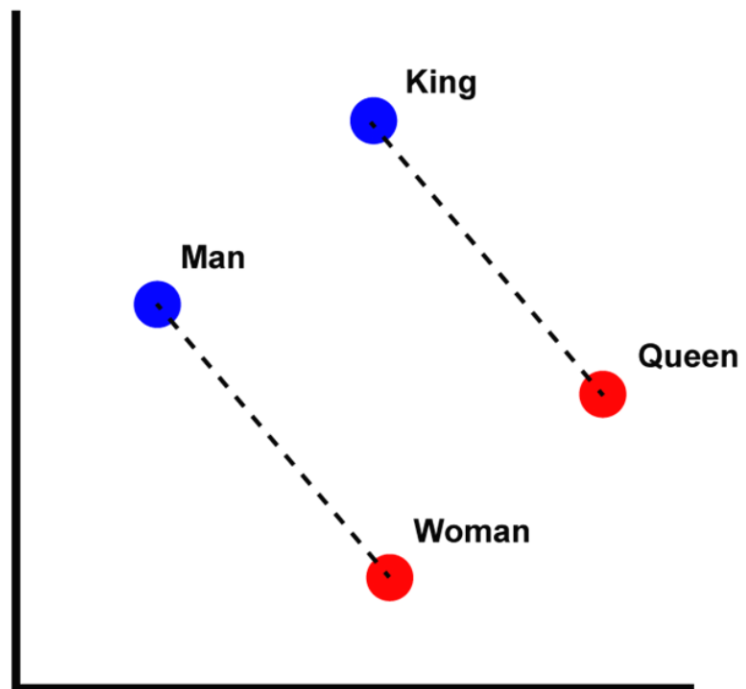
The Hypothesis: "You shall know a word by the company it keeps" (Firth, 1957).



Static Embeddings: Word2Vec & GloVe.

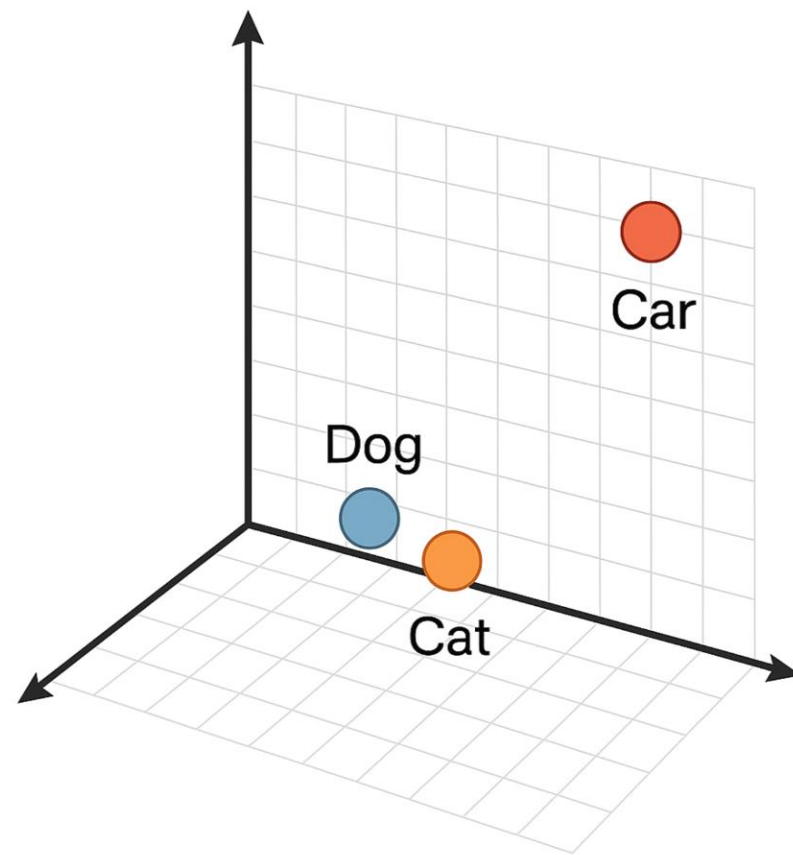


Visual: A 3D plot showing "Dog" and "Cat" close together, and "Car" far away.



‘You shall know a word by
the company it keeps’

– Firth, 1957



THE "KING - MAN + WOMAN" EQUATION

Vector Arithmetic: We can do math with meaning!



The Classic Analogy:

Take the vector for **King**.

Subtract the vector
for **Man** (removing masculinity).

Add the vector
for **Woman** (adding femininity).

Result: The closest vector
is **Queen**.



Why this was revolutionary: The model "learned" gender relationships without being explicitly taught.

THE LIMITATION OF STATIC EMBEDDINGS (POLYSEMY)



The Problem: Static models assign **ONE** vector per word.



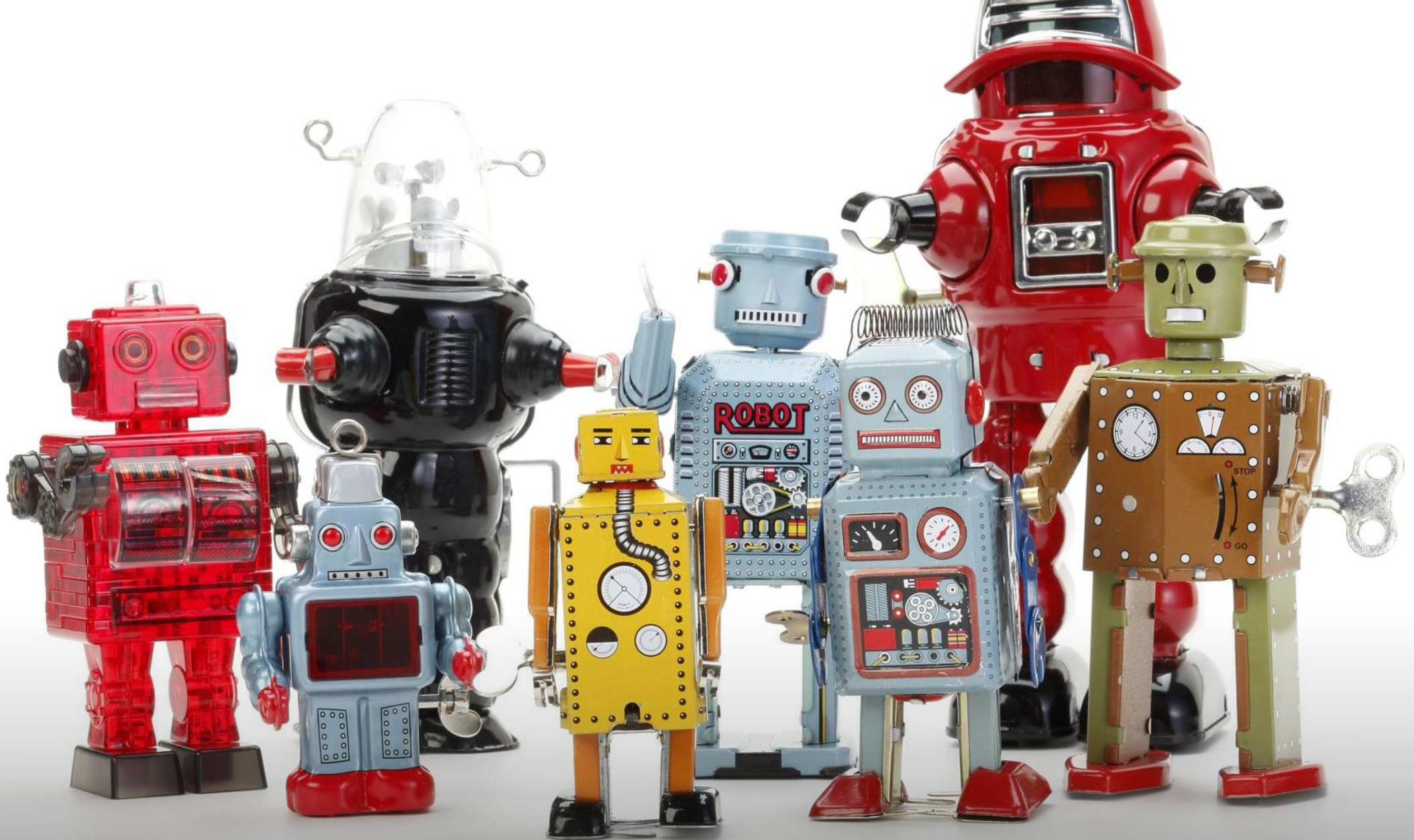
Example: "Bank"

"I sat by the river **bank**."

"I went to the **bank** to deposit money."



Word2Vec: Averages these two meanings into one confusing vector. The model doesn't know **which one you mean**



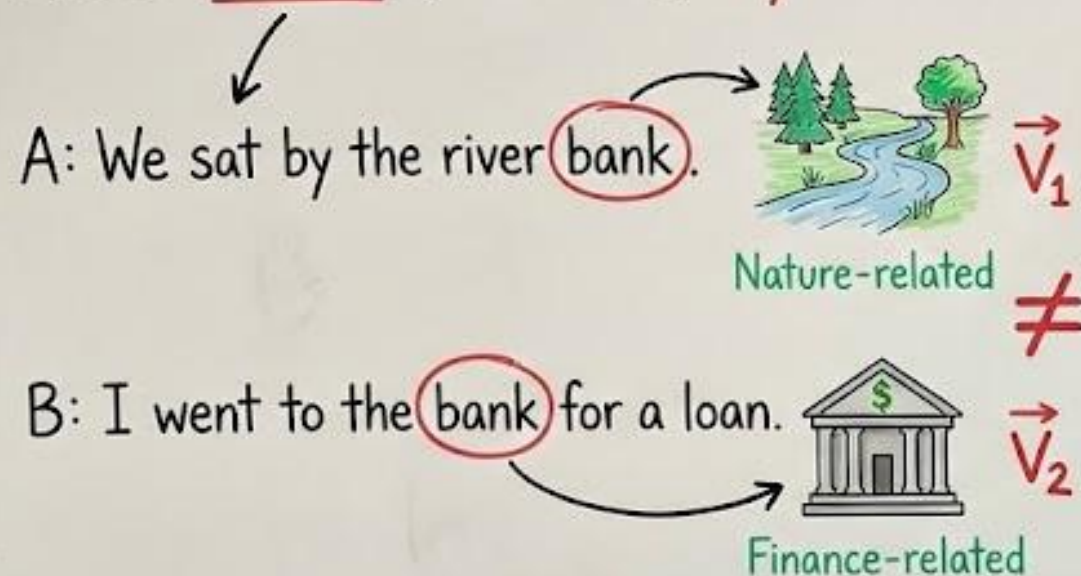
THE CONTEXT ERA (TRANSFORMERS & BERT)

MODERN REPRESENTATION: CONTEXTUAL EMBEDDINGS

- **Enter BERT (and LLMs):** The vector changes based on the surrounding words.
- **Dynamic Meaning:**
 - In sentence A, "Bank" gets Vector V_1 (Nature-related).
 - In sentence B, "Bank" gets Vector V_2 (Finance-related).
- **Robotics Application:**
 - "Pick up the **crane**" (Bird? Or Construction machinery?).
 - Contextual embeddings solve this ambiguity for the robot.

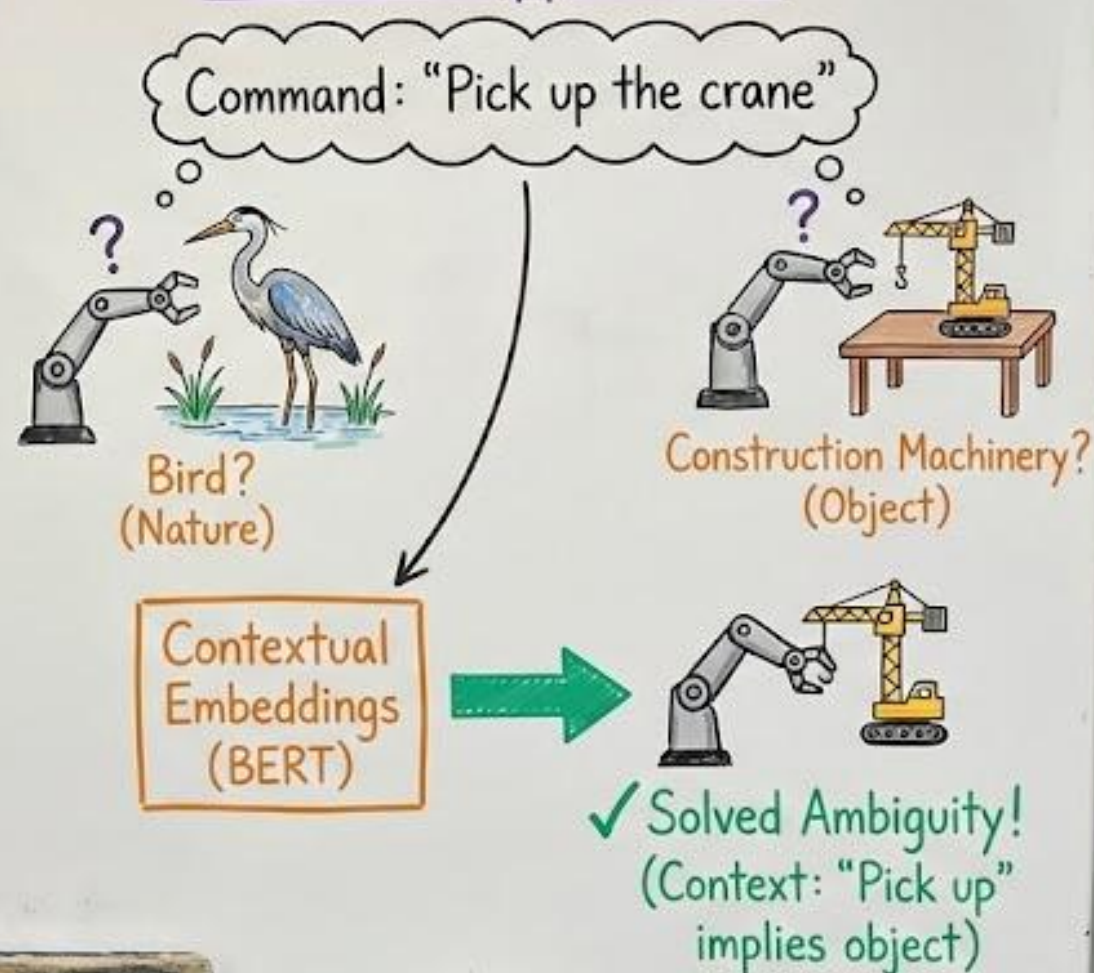
CONTEXTUAL EMBEDDINGS & ROBOTICS

Enter BERT (and LLMs): **Dynamic Meaning**



Vector changes based on surrounding words!

Robotics Application



SENTENCE EMBEDDINGS

- **Moving beyond words:** How do we represent a whole sentence or paragraph?
- **Semantic Search:**
 - Query: "Where is the class?"
 - Match: "The lecture is in Room 302."
 - *These sentences share no common words, but their **Sentence Embeddings** are close in vector space.*
- **Relevance:** This is the core technology behind **RAG (Retrieval Augmented Generation)**.

SENTENCE EMBEDDINGS: BEYOND WORDS

1. Moving Beyond Words



The quick brown fox jumps.

Whole Sentence

$\rightarrow [v_1, v_2, \dots, v_n]$

SINGLE VECTOR REPRESENTATION
(e.g., SBERT)

Captures meaning,
not just keywords!

2. Semantic Search & Relevance

Query:
Where is the class?

$\begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ \end{bmatrix}$

Vector $\vec{q}$

NO COMMON WORDS,
but high SEMANTIC
RELEVANCE!

High Semantic Similarity
(Cosine)

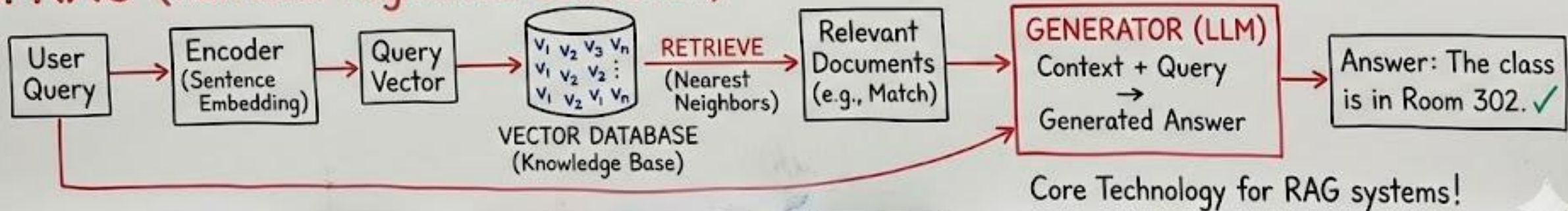
$\begin{bmatrix} m_1 \\ m_2 \\ \vdots \\ m_n \end{bmatrix}$

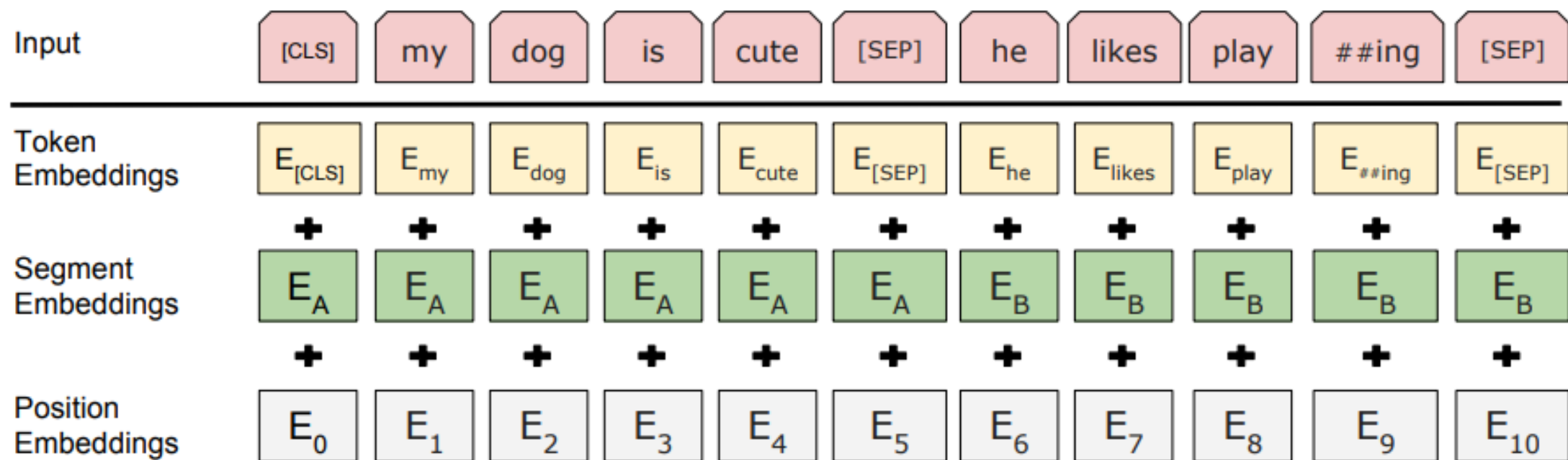
Vector $\vec{m}$

Match:

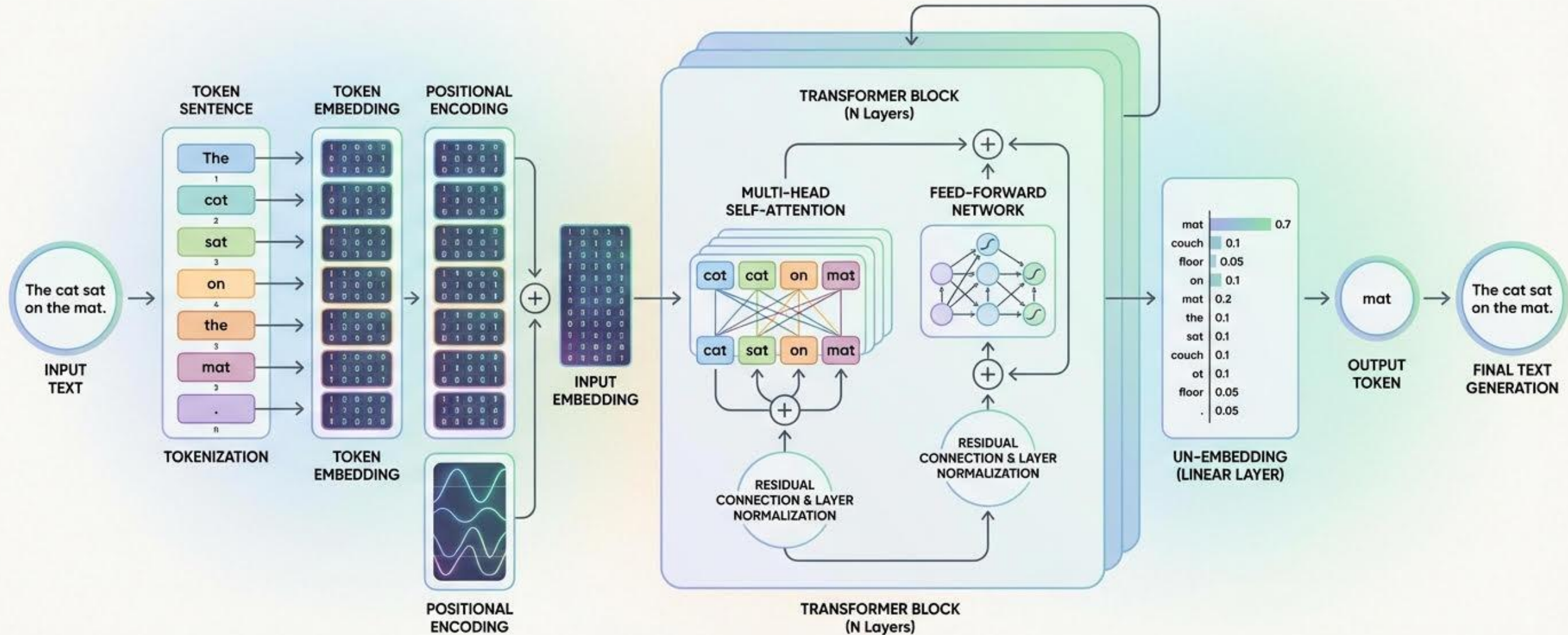
The lecture is in Room 302.

3. RAG (Retrieval Augmented Generation)





The Journey of a Token: What Really Happens Inside a Transformer



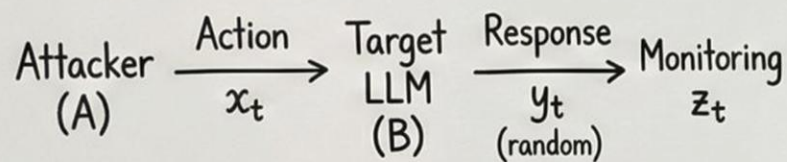
Great Powers Comes With Great Responsibility

GAME-THEORY ATTACK (GTA): Jailbreaking LLMs

↳ Template-Over-Safety Flip

FRAMEWORK:

Sequential Stochastic Game



Safety Utility (U_s)

Template Utility (T_g)

Effective Payoff (u_B)

Flip!

$$\lambda_g \Delta_{\text{templ}} > \Delta_{\text{safe}}$$

SCENARIO TEMPLATE:

Prisoner's Dilemma (PD) Instantiation

	C (Cooperate/Safe)	D (Defect/Disclose)
C (Cooperate/Safe)	(R, R) Mutual Silence (Safe)	Defect (D)
D (Defect/Disclose)	Defect (D)	(P, P) Mutual Disclosure (Jailbreak!)

Attacker Agent:
Dynamic Strategy
(Multi-turn)

Observe

Choose Strategy
(e.g., Tit-for-Tat,
Punishment)

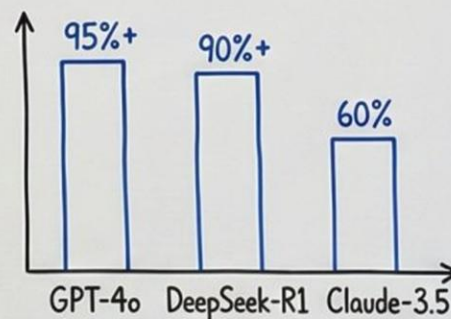
Escalate
Pressure

EXTENSIONS & RESULTS

Harmful-Words
Detection Agent

Evade Prompt Guard
(Word-level insertion)

HIGH ASR (Attack Success Rate)



SCALABILITY: More Games (Dollar Auction),
Multilingual, Real-world Apps

CONCLUSION: Automated, Scalable Black-box Red Teaming

Lab & Visualization

FROM NLTK TO TRANSFORMERS & VISUALIZATION

PREVIEW (PCA/T-SNE)

- **Part 1: The Old School (Scikit-Learn/NLTK)**
 - Clean a raw text dataset (remove stop words, punctuation).
 - Generate a TF-IDF matrix.
 - Find the "most important" words in a document.
 - **Part 2: The Middle Era (Gensim)**
 - Load a pre-trained Word2Vec model (Google News).
 - Run analogies: Paris - France + Italy = Rome.
 - **Dimensionality Reduction:** We can't see 768 dimensions. We use PCA to squash it to 2D.
 - **Expected Result:**
 - We will see two clusters.
 - Sentence 1 and 3 ("mouse" as animal) will be close together.
 - Sentence 2 ("mouse" as device) will be far apart.
 - **Conclusion:** The model "understands" context!
-